

Securing and Scaling a Production-Style Web Infrastructure

*NGINX Reverse Proxy & Load Balancing · TLS/HTTPS Encryption · UFW Firewall Hardening ·
Fault Diagnosis on Parrot OS*

Technical Report

Module: Web Infrastructure
Group: Group 07
Host System: Parrot Security OS 7.3 (kernel 7.0.13-parrot7-amd64)
Environment: Single Linux VM — NGINX, UFW, self-signed TLS
Submitted by: Jacinta Ngugi
Date: 21 September 2026

Link to the video:
<https://drive.google.com/file/d/1VF7UpZ5UFVg3mfAHzFNBtSp4oV1RMC4h/view?usp=drivesdk>

1. Architecture Overview

This project implements a small, production-style web deployment on a single Parrot OS host. Clients never reach the application backends directly: every request enters through NGINX, which terminates TLS, enforces an HTTP-to-HTTPS redirect, and load-balances traffic across two independent backend services. A host firewall (UFW) restricts inbound access to only the ports required for administration and for the public web service.

```
CLIENT
| HTTP:80 --> 301 redirect --> HTTPS:443
v
FIREWALL (UFW) -- allow 22, 80, 443 / deny 3001, 3002
v
NGINX -- reverse proxy + load balancer + TLS termination
|
+-----+
v       v
Backend 1 (127.0.0.1:3001) Backend 2 (127.0.0.1:3002)
"Web Infrastructure - Group_07 - Server 1" "...Server 2"
```

Backend services bind only to the loopback interface 127.0.0.1, so they are unreachable from outside the host even before the firewall is considered — this is the first layer of defense-in-depth. UFW then

removes the possibility of exposure on those ports entirely, and NGINX's upstream block is the only component permitted to speak to them.

2. Environment Preparation

The host was updated and the required packages installed: nginx, ufw, curl, openssl, python3, openssh-server and zip. The NGINX and SSH services were enabled to start automatically via systemd, and both were confirmed active. A snapshot of the environment (OS release, kernel version, NGINX, OpenSSL, Python and UFW versions) was captured to environment_versions.txt for inclusion as evidence, since the exact software versions affect how each configuration directive behaves.

```
leo_01@parrot:~$ sudo apt install -y nginx ufw curl openssl python3 openssh-server zip
(sudo) password for leo_01:
nginx is already the newest version (1.26.3-3+deb13u9).
nginx set to manually installed.
ufw is already the newest version (0.36.2-9).
curl is already the newest version (8.21.0-2-bpo13+1).
openssl is already the newest version (3.5.7-1-deb13u2).
python3 is already the newest version (3.13.6-1).
python3 set to manually installed.
openssh-server is already the newest version (1:10.3p1-1-bpo13+1).
zip is already the newest version (3.0-13+deb13u1).
zip set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 28
leo_01@parrot:~$ sudo systemctl enable --now nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
leo_01@parrot:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
leo_01@parrot:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: 1)
   Active: active (running) since Fri 2026-09-18 18:05:46 +04; 2 days ago
     Invocation: 34f0235d92e943cab8d83c5d86ae515
       Docs: man:nginx(8)
    Main PID: 230615 (nginx)
       Tasks: 13 (limit: 14537)
      Memory: 10M (peak: 28.6M)
         CPU: 250ms
    CGroup: /system.slice/nginx.service
            └─230615 "nginx: master process /usr/sbin/nginx -g daemon on: m
            └─230617 "nginx: worker process"
            └─230618 "nginx: worker process"
            └─230619 "nginx: worker process"
            └─230620 "nginx: worker process"
            └─230621 "nginx: worker process"
            └─230622 "nginx: worker process"
            └─230623 "nginx: worker process"
            └─230624 "nginx: worker process"
            └─230625 "nginx: worker process"
            └─230626 "nginx: worker process"
            └─230627 "nginx: worker process"
            └─230628 "nginx: worker process"

Sep 20 12:27:56 parrot systemd[1]: nginx.service: Control process exited, ee
Sep 20 12:27:56 parrot systemd[1]: Reload failed for nginx.service - A high
Sep 20 12:29:15 parrot systemd[1]: Reloading nginx.service - A high perform
Sep 20 12:29:15 parrot systemd[1]: nginx.service: Control process exited, ee
Sep 20 12:29:15 parrot systemd[1]: Reload failed for nginx.service - A high
Sep 20 12:31:59 parrot systemd[1]: Reloading nginx.service - A high perform
Sep 20 12:31:59 parrot systemd[1]: nginx.service: Control process exited, ee
Sep 20 12:31:59 parrot systemd[1]: Reload failed for nginx.service - A high
leo_01@parrot:~$ systemctl status ssh
```

Figure 1: Package installation, and NGINX/SSH enabled and confirmed active via systemctl status.

```
leo_01@parrot:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-09-16 09:25:49 +04; 5 days ago
     Invocation: 4262166c0a974509ba71e53a92cb2a39
       Docs: man:ssh(8)
            man:ssh-config(5)
    Main PID: 1213 (sshd)
      Tasks: 1 (limit: 16537)
     Memory: 1.8M (peak: 2.8M)
        CPU: 29ms
    CGroup: /system.slice/ssh.service
            └─1213 'sshd: /usr/sbin/sshd -D [listener] 0 of 16-100 startups'

Sep 16 09:25:49 parrot systemd[1]: Starting ssh.service - OpenBSD Secure Shell server:
Sep 16 09:25:49 parrot sshd[1213]: Server listening on 0.0.0.0 port 22.
Sep 16 09:25:49 parrot sshd[1213]: Server listening on *: port 22.
Sep 16 09:25:49 parrot systemd[1]: Started ssh.service - OpenBSD Secure Shell server:
leo_01@parrot:~$ cat /etc/os-release
echo '=== OS ==='
echo 'cat /etc/os-release'
echo '=== Kernel ==='
uname -r
echo '=== NGINX ==='
nginx -v
echo '=== OpenSSL ==='
openssl version
echo '=== Python ==='
python3 --version
echo '=== UFW ==='
ufw --version 2>&1
$ > environment_versions.txt
leo_01@parrot:~$ cat environment_versions.txt
=== OS ===
PRETTY_NAME="Parrot Security 7.3 (echo)"
NAME="Parrot Security"
VERSION_ID="7.3"
VERSION="7.3 (echo)"
VERSION_CODENAME=echo
DEBIAN_VERSION_FULL=13.5
ID=parrot
ID_LIKE=debian
HOME_URL="https://www.parrotsec.org/"
SUPPORT_URL="https://www.parrotsec.org/docs"
BUG_REPORT_URL="https://github.com/parrotsec/"

=== Kernel ===
7.0.13-parrot7-amd64

=== NGINX ===
bash: nginx: command not found

=== OpenSSL ===
OpenSSL 3.5.7 9 Jun 2026 (Library: OpenSSL 3.5.7 9 Jun 2026)

=== Python ===
```

Figure 2: SSH service active and listening on port 22; `environment_versions.txt` generated, recording the Parrot OS release, kernel, NGINX, OpenSSL, Python and UFW versions used throughout the exercise.

3. Part A — Backend Environment

Two minimal Python HTTP services were created, each returning a distinguishable plain-text response so that NGINX's load-balancing behaviour can be proven later. Backend 1 listens on 127.0.0.1:3001 and returns “Web Infrastructure - Group_07 - Server 1”; Backend 2 listens on 127.0.0.1:3002 and returns the equivalent “Server 2” string. Both are managed as systemd services (`backend1.service` / `backend2.service`) running as the unprivileged `www-data` user with `Restart=on-failure`, so a crashed backend recovers automatically.

```

=== Python ===
Python 3.13.5

=== UFW ===
bash: ufw: command not found
leo_01@parrot:~$ mkdir -p ~/Group_07_SecureWebInfrastructures/(Demo,Architecture,NGI,NX,Backend/backend1,Backend/backend2,TLS,Firewall,Evidence/
andeng,HTTPS,Firewall,Troubleshooting)
leo_01@parrot:~$ cd Group_07_SecureWebInfrastructures/
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ mv ~/environment_versions.txt Evidence/
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l Evidence/
total 4
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Backend
-rw-rw-r-- 1 leo_01 leo_01 548 Sep 21 10:27 environment_versions.txt
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Firewall
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 HTTPS
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Load_Balancing
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Troubleshooting
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls
Architecture Backend Demo Evidence Firewall NGI NX TLS
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l
total 0
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Architecture
drwxrwxr-x 1 leo_01 leo_01 32 Sep 21 10:29 Backend
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Demo
drwxrwxr-x 1 leo_01 leo_01 146 Sep 21 10:30 Evidence
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 Firewall
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 NGI NX
drwxrwxr-x 1 leo_01 leo_01 0 Sep 21 10:29 TLS
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo mkdir -p /opt/backend1
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo tee /opt/backend1/server.py > /dev/null <<'PY'
from http.server import BaseHTTPRequestHandler, HTTPServer

RESPONSE = "Web Infrastructure - Group_07 - Server 1\n"

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = RESPONSE.encode()
        self.send_response(200)
        self.send_header('Content-Type', 'text/plain; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

if __name__ == '__main__':
    HTTPServer(("127.0.0.1", 3001), Handler).serve_forever()
PY
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo mkdir -p /opt/backend2
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo tee /opt/backend2/server.py > /dev/null <<'PY'
from http.server import BaseHTTPRequestHandler, HTTPServer

RESPONSE = "Web Infrastructure - Group_07 - Server 2\n"

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = RESPONSE.encode()
        self.send_response(200)
        self.send_header('Content-Type', 'text/plain; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

```

Figure 3: Group submission folder structure created, and the Backend 1 Python HTTP handler (server.py) written to /opt/backend1, returning the Server 1 response on port 3001.

```

if __name__ == '__main__':
    HTTPServer(("127.0.0.1", 3002), Handler).serve_forever()
PY
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/systemd/system/
total 64
drwxr-xr-x 1 root root 34 Jun 25 22:45 bluetooth.target.wants
drwxr-xr-x 1 root root 46 Jun 25 22:45 dbus-fi.wl.wpa_supplicant1.service -> /usr/lib/systemd/system/wpa_supplicant.service
lrwxrwxrwx 1 root root 41 Jun 25 22:45 dbus-org.bluez.service -> /usr/lib/systemd/system/bluetooth.service
lrwxrwxrwx 1 root root 46 Jun 25 22:45 dbus-org.freedesktop.ModemManager1.service -> /usr/lib/systemd/system/ModemManager.service
lrwxrwxrwx 1 root root 57 Jun 25 22:45 dbus-org.freedesktop.nm-dispatcher.service -> /usr/lib/systemd/system/NetworkManager-dispatcher.servi
lrwxrwxrwx 1 root root 51 Jun 25 22:45 dbus-org.rnd2.cuppowei.gul.helper.service -> /usr/lib/systemd/system/cuppowei.gul.helper.service
drwxr-xr-x 1 root root 54 Jun 25 22:45 default.target.wants
lrwxrwxrwx 1 root root 35 Jun 25 22:45 display-manager.service -> /lib/systemd/system/lightdm.service
drwxr-xr-x 1 root root 66 Jun 25 22:45 firewall.service.wants
drwxr-xr-x 1 root root 36 Jun 15 13:16 getty.target.wants
drwxr-xr-x 1 root root 186 Jun 25 22:45 graphical.target.wants
drwxr-xr-x 1 root root 40 Jun 25 22:45 hibernate.target.wants
drwxr-xr-x 1 root root 40 Jun 25 22:45 hybrid-sleep.target.wants
lrwxrwxrwx 1 root root 29 Jun 25 22:45 iofwd.service -> /dev/null
lrwxrwxrwx 1 root root 42 Sep 18 15:00 iscsi.service -> /usr/lib/systemd/system/open-iscsi.service
drwxr-xr-x 1 root root 128 Jun 25 22:45 mdmonitor.service.wants
drwxr-xr-x 1 root root 168 Sep 20 11:17 multi-user.target.wants
drwxr-xr-x 1 root root 104 Jun 25 22:45 network-online.target.wants
drwxr-xr-x 1 root root 60 Jun 25 22:45 network-pre.target.wants
lrwxrwxrwx 1 root root 38 Aug 27 21:42 ntpd.service -> /usr/lib/systemd/system/ntpsec.service
lrwxrwxrwx 1 root root 38 Aug 27 21:42 ntp.service -> /usr/lib/systemd/system/ntpsec.service
drwxr-xr-x 1 root root 28 Jun 25 22:45 open-vm-tools.service.requires
lrwxrwxrwx 1 root root 45 Jun 25 22:45 smartd.service -> /usr/lib/systemd/system/smartmontools.service
-rw-r--r-- 1 root root 326 Aug 19 15:45 snap-core18-2999.mount
-rw-r--r-- 1 root root 326 Sep 13 00:27 snap-core18-3084.mount
drwxr-xr-x 1 root root 220 Sep 13 00:27 snapd.mounts.target.wants
-rw-r--r-- 1 root root 326 Aug 19 15:45 snap-postman-560.mount
-rw-r--r-- 1 root root 326 Aug 19 15:45 snap-snapd-27710.mount
-rw-r--r-- 1 root root 326 Sep 7 08:08 snap-snapd-27738.mount
drwxr-xr-x 1 root root 412 Sep 18 18:03 sockets.target.wants
lrwxrwxrwx 1 root root 35 Aug 19 10:22 sshd.service -> /usr/lib/systemd/system/ssh.service
drwxr-xr-x 1 root root 38 Jun 25 22:45 sshd.service.wants
drwxr-xr-x 1 root root 38 Jun 25 22:45 ssh.service.wants
drwxr-xr-x 1 root root 38 Jun 25 22:45 ssh.socket.wants
drwxr-xr-x 1 root root 40 Jun 25 22:45 suspend.target.wants
drwxr-xr-x 1 root root 40 Jun 25 22:45 suspend-then-hibernate.target.wants
drwxr-xr-x 1 root root 314 Sep 18 18:03 systemd.target.wants
drwxr-xr-x 1 root root 124 Jun 25 22:45 systemd.service.wants
drwxr-xr-x 1 root root 70 Jun 25 22:45 systemd-coredump.service.wants
drwxr-xr-x 1 root root 44 Jun 25 22:45 systemd.slice.wants
drwxr-xr-x 1 root root 492 Jun 25 22:45 timers.target.wants
lrwxrwxrwx 1 root root 45 Jun 25 22:45 vmtoolsd.service -> /usr/lib/systemd/system/open-vm-tools.service
-rw-r--r-- 1 root root 298 Sep 20 11:38 webinfra-backend1.service
-rw-r--r-- 1 root root 298 Sep 20 11:38 webinfra-backend2.service
lrwxrwxrwx 1 root root 47 Sep 20 10:53 webinfra-backend1.service -> /etc/systemd/system/webinfra-backend1.service
-rw-r--r-- 1 root root 0 Sep 18 21:13 webinfra-backend2.service
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo tee /etc/systemd/system/backend1.service > /dev/null <<'EOF'
[Unit]
Description=Web Infrastructure Backend 1
After=network.target

[Service]
Type=simple
User=www-data

```

Figure 4: Backend 2 handler written to /opt/backend2 (port 3002), and the systemd unit file for Backend 2 created with Restart=on-failure and User=www-data.

```

User=www-data
Group=www-data
ExecStart=/usr/bin/python3 /opt/backend1/server.py
Restart=on-failure
RestartSec=2

[Install]
WantedBy=multi-user.target
EOF
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo systemctl enable --now backend1.service
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo systemctl status backend1.service
* backend1.service - Web Infrastructure Backend 1
   Loaded: loaded (/etc/systemd/system/backend1.service; enabled; preset: [ ]
   Active: active (running) since Mon 2026-09-21 10:36:33 +04: 51ms ago
   Invocation: 367b41c6093e4c558465d7fd817b62e0
   Main PID: 535519 (python3)
   Tasks: 1 (limit: 18537)
   Memory: 8M (peak: 8M)
   CPU: 45ms
   CGroup: /system.slice/backend1.service
           └─535519 /usr/bin/python3 /opt/backend1/server.py

Sep 21 10:36:33 parrot systemd[1]: backend1.service: Scheduled restart job,
Sep 21 10:36:33 parrot systemd[1]: Started backend1.service - Web Infrastru
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ tput setaf 14
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo systemctl status backend2.service
* backend2.service - Web Infrastructure Backend 2
   Loaded: loaded (/etc/systemd/system/backend2.service; enabled; preset: [ ]
   Active: active (running) since Mon 2026-09-21 10:37:56 +04: 1min 1s ago
   Invocation: ccc89045905472d67e0d9f11c549592
   Main PID: 535444 (python3)
   Tasks: 1 (limit: 18537)
   Memory: 8.5M (peak: 9.3M)
   CPU: 88ms
   CGroup: /system.slice/backend2.service
           └─535444 /usr/bin/python3 /opt/backend2/server.py

Sep 21 10:37:56 parrot systemd[1]: Started backend2.service - Web Infrastru
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ tput setaf 14
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo ss -tlnp | grep -E ':3001|:3002'
LISTEN 0      5            127.0.0.1:3001      0.0.0.0:*        users:((('python3',pid=420295,fd=33))

```

Figure 5: Both `backend1.service` and `backend2.service` enabled and active (running); `ss -tlnp` confirms both processes are listening only on the 127.0.0.1 loopback address on ports 3001 and 3002.

```

LISTEN 0      5            127.0.0.1:3002      0.0.0.0:*        users:((('python3',pid=535444,fd=33))
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -s http://127.0.0.1:3001
Web Infrastructure - Group_07 - Server 1
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -s http://127.0.0.1:3002
Web Infrastructure - Group_07 - Server 2
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ ss -tlnp | grep -E ':3001|:3002' > Evidence/Backend/listening_ports.txt
systemctl is-active backend1.service backend2.service > Evidence/Backend/service_state.txt
curl -s http://127.0.0.1:3001 > Evidence/Backend/backend1_response.txt
curl -s http://127.0.0.1:3002 > Evidence/Backend/backend2_response.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ cp /opt/backend1/server.py Backend/backend1/
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ cat Backend/backend1
cat: Backend/backend1: is a directory
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ cat Backend/backend1/server.py
from http.server import BaseHTTPRequestHandler, HTTPServer

RESPONSE = 'Web Infrastructure - Group_07 - Server 1\n'

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = RESPONSE.encode()
        self.send_response(200)
        self.send_header('Content-Type', 'text/plain; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

if __name__ == '__main__':
    HTTPServer((127.0.0.1, 3001), Handler).serve_forever()
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ cat Backend/backend2/server.py
from http.server import BaseHTTPRequestHandler, HTTPServer

RESPONSE = 'Web Infrastructure - Group_07 - Server 2\n'

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        body = RESPONSE.encode()
        self.send_response(200)
        self.send_header('Content-Type', 'text/plain; charset=utf-8')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers()
        self.wfile.write(body)

if __name__ == '__main__':
    HTTPServer((127.0.0.1, 3002), Handler).serve_forever()
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo chown -R USER:USER ~/Group_07_SecureWebInfrastructure
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo tee /etc/nginx/sites-available/group07.conf > /dev/null <<'EOF'
upstream backend_pool
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}

server {
    listen 80;
    server_name localhost;

```

Figure 6: Direct `curl` tests confirm each backend returns its own distinct response; service state, listening ports and both backend responses saved as evidence; backend scripts copied into the submission folder and the initial NGINX upstream (`backend_pool`) configuration created.

4. Part B — NGINX Reverse Proxy and Load Balancing

NGINX was configured with an upstream block named `backend_pool` containing both backend addresses, and a server block that proxies all requests to that pool. This is the core of the reverse-proxy

design: clients only ever address NGINX, and NGINX decides which backend actually serves each request using its default round-robin algorithm.

```
upstream backend_pool {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}
location / {
    proxy_pass http://backend_pool;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Every configuration change was validated with `nginx -t` before being applied with `systemctl reload nginx`, avoiding downtime from a bad configuration. Sending ten sequential requests to NGINX produced alternating Server 1 / Server 2 responses, proving that both backends actively share the load. Stopping `backend1.service` temporarily and repeating the test produced only Server 2 responses, confirming NGINX correctly routes around an unavailable backend; restarting the service returned traffic to a balanced pattern.

The terminal screenshot shows a series of commands and their outputs. It starts with a `systemctl reload nginx` command, which outputs status information for the `nginx.service`, including its state as 'active (running)' and its configuration files. This is followed by a loop of `do curl -s http://127.0.0.1;` commands, which return alternating responses from 'Server 1' and 'Server 2'. Then, the `backend1.service` is stopped using `systemctl stop backend1.service`. Another loop of `do curl -s http://127.0.0.1;` commands follows, showing that all requests now come from 'Server 2'. Finally, `backend1.service` is restarted with `systemctl start backend1.service`.

Figure 7: NGINX reloaded and active; ten repeated requests to NGINX alternate between “Server 1” and “Server 2”, providing load balancing across the backend pool. Backend 1 was then stopped, repeat requests, and restarted to confirm NGINX continues serving from the remaining backend during an outage.

5. Troubleshooting Case 1 — NGINX Failed to Load Its Configuration

5.1 Symptom

While enabling the new site configuration, NGINX began failing to reload. `systemctl` reported repeated “Reload failed for nginx.service” events, and `nginx -t` returned an emerg-level error indicating it could not find the main configuration file: “No such file or directory in /etc/nginx/nginx.conf:1”.

```
location / {
    proxy_pass http://backend_pool;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

EOF
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ln -sf /etc/nginx/sites-available/group07.conf /etc/nginx/sites-enabled/group07.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/sites-enabled/default
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
2026/09/21 10:47:06 (emerg) 536571#536571: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
nginx: configuration file /etc/nginx/nginx.conf test failed
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/sites-enabled/webinfra
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
2026/09/21 10:47:34 (emerg) 536943#536943: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
nginx: configuration file /etc/nginx/nginx.conf test failed
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/nginx/sites-enabled/
total 8
-rwxrwxrwx 1 root root 44 Sep  5 21:52 group-site -> /etc/nginx/sites-available/webinfrastructure
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/sites-enabled/group-site
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ls -l /etc/nginx/sites-enabled/
total 4
-rwxrwxrwx 1 root root 39 Sep 21 10:46 group07.conf -> /etc/nginx/sites-available/group07.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/nginx/sites-enabled/group07.conf
-rwxrwxrwx 1 root root 39 Sep 21 10:46 /etc/nginx/sites-enabled/group07.conf -> /etc/nginx/sites-available/group07.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
2026/09/21 10:49:53 (emerg) 537306#537306: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
nginx: configuration file /etc/nginx/nginx.conf test failed
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/nginx/nginx.conf:1
ls: cannot access '/etc/nginx/nginx.conf:1': No such file or directory
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ cat /etc/nginx/nginx.conf:1
cat: /etc/nginx/nginx.conf:1: No such file or directory
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ls -l /etc/nginx/sites-available/
total 20
-rw-r--r-- 1 root root 2412 Jun  5 16:22 default
-rw-r--r-- 1 root root 405 Sep 21 10:45 group07.conf
-rw-r--r-- 1 root root 2017 Sep 20 12:22 webinfra
-rw-r--r-- 1 root root 1320 Sep 10 12:41 webinfrastructure
-rw-r--r-- 1 root root 1320 Sep  5 22:21 webinfrastructure.bak
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/sites-available/webinfra
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ls -l /etc/nginx/sites-available/
total 16
-rw-r--r-- 1 root root 2412 Jun  5 16:22 default
-rw-r--r-- 1 root root 405 Sep 21 10:45 group07.conf
-rw-r--r-- 1 root root 1320 Sep 10 12:41 webinfrastructure
-rw-r--r-- 1 root root 1320 Sep  5 22:21 webinfrastructure.bak
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
2026/09/21 10:51:41 (emerg) 537607#537607: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
nginx: configuration file /etc/nginx/nginx.conf test failed
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nano /etc/nginx/conf.d/webinfra_log_format.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/conf.d/webinfra_log_format.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo cat /etc/nginx/conf.d/webinfra_log_format.conf
cat: /etc/nginx/conf.d/webinfra_log_format.conf: No such file or directory
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ls -ls /etc/nginx/sites-available/group07.conf /etc/nginx/sites-enabled/group07.conf
4 -rwxrwxrwx 1 root root 44 Sep  5 21:52 /etc/nginx/sites-available/group07.conf
4 -rwxrwxrwx 1 root root 44 Sep  5 21:52 /etc/nginx/sites-enabled/group07.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
2026/09/21 10:55:05 (emerg) 538551#538551: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
```

Figure 8: `nginx -t` fails with an emerg error referencing `/etc/nginx/nginx.conf:1`; repeated attempts to remove and relink the `sites-available/sites-enabled` configuration, and directory listings taken while narrowing down the fault.

```
2026/09/21 10:55:05 (emerg) 538551#538551: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
nginx: configuration file /etc/nginx/nginx.conf test failed
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo apt update
Hit:1 https://packages.microsoft.com/repos/code stable InRelease
Hit:2 https://deb.parrot.ch/parrot echo InRelease
Hit:3 https://deb.parrot.ch/direct/parrot echo-backports InRelease
Hit:4 https://deb.parrot.ch/parrot echo-backports InRelease
Hit:5 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Hit:6 https://us-central1-apt.pkg.dev/projects/antigravity-auto-updater-dev antigravity-debian InRelease
07 packages can be upgraded. Run 'apt list --upgradable' to see them.
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo apt upgrade -y
APT on Parrot behaves differently than Debian.
apt upgrade is equivalent to apt full-upgrade in Debian,
and performs a complete system update.

Use apt safe-upgrade to perform a partial upgrade.

Upgrading:
bind9-dnsutils google-chrome-stable python3-tlsoc
bind9-host libgmplib3-0 python3-fdb
bind9-libs libldb2 samba
bind9-dnsutils-dkms libnss-wslbind samba-common
code libpam-wslbind samba-common-bin
file-roller libnss-wslbind samba-libs
gimp libldb1 smbclient
gimp-data libldb1 tdb-tools
git-1.2-gimp-3.0 libevent0t64 tor
golang-1.25 libbuntdb5 tor-gui-pdb
golang-1.25-doc libbwt0t0t0 wlnbind
golang-1.25-go python3-ldb
golang-1.25-src python3-samba

Summary:
Upgrading: 37, Installing: 0, Removing: 0, Not Upgrading: 0
Download size: 836 MB
Space needed: 4,258 kB / 10.4 GB available

Continue? (Y/n) Y
Get:1 https://packages.microsoft.com/repos/code stable/main amd64 code amd64 1.138.0-1789458741 (238 MB)
Get:2 https://dl.google.com/linux/chrome-stable/deb stable/main amd64 google-chrome-stable amd64 133.0.8010.52-1 (142 MB)
Get:3 https://deb.parrot.ch/parrot echo-backports/main amd64 libnss-wslbind amd64 2:4.24.7-dfsg-1-bpo13+1 (129.7 kB)
Get:4 https://deb.parrot.ch/parrot echo-backports/main amd64 libpam-wslbind amd64 2:4.24.7-dfsg-1-bpo13+1 (132 kB)
Get:5 https://deb.parrot.ch/parrot echo-backports/main amd64 samba-common-all 2:4.24.7-dfsg-1-bpo13+1 (68.6 kB)
Get:6 https://deb.parrot.ch/parrot echo-backports/main amd64 wlnbind amd64 2:4.24.7-dfsg-1-bpo13+1 (1436 kB)
Get:7 https://deb.parrot.ch/parrot echo-backports/main amd64 libbwt0t0t0 amd64 2:4.24.7-dfsg-1-bpo13+1 (124.0 kB)
Get:8 https://deb.parrot.ch/parrot echo-backports/main amd64 libldb1 amd64 2:2.4.4+samba4.24.7-dfsg-1-bpo13+1 (65.7 kB)
Get:9 https://deb.parrot.ch/parrot echo-backports/main amd64 libevent0t64 amd64 2.0.17.1+samba4.24.7-dfsg-1-bpo13+1 (80.8 kB)
Get:10 https://deb.parrot.ch/parrot echo-backports/main amd64 libbuntdb5 amd64 2:4.24.7-dfsg-1-bpo13+1 (103 kB)
Get:11 https://deb.parrot.ch/parrot echo-backports/main amd64 smbclient amd64 2:4.24.7-dfsg-1-bpo13+1 (485 kB)
Get:12 https://deb.parrot.ch/parrot echo-backports/main amd64 libldb1 amd64 2:1.4.16+samba4.24.7-dfsg-1-bpo13+1 (85.4 kB)
Get:13 https://deb.parrot.ch/parrot echo-backports/main amd64 python3-samba amd64 2:4.24.7-dfsg-1-bpo13+1 (2,921 kB)
Get:14 https://deb.parrot.ch/parrot echo-backports/main amd64 samba amd64 2:4.24.7-dfsg-1-bpo13+1 (1,225 kB)
Get:15 https://deb.parrot.ch/parrot echo-backports/main amd64 samba-common-bin amd64 2:4.24.7-dfsg-1-bpo13+1 (1,379 kB)
Get:16 https://deb.parrot.ch/parrot echo-backports/main amd64 python3-ldb amd64 2:2.11.0+samba4.24.7-dfsg-1-bpo13+1 (80.1 kB)
Get:17 https://deb.parrot.ch/parrot echo-backports/main amd64 libldb2 amd64 2:2.11.0+samba4.24.7-dfsg-1-bpo13+1 (112 kB)
Get:18 https://deb.parrot.ch/parrot echo-backports/main amd64 samba-libs amd64 2:4.24.7-dfsg-1-bpo13+1 (6,086 kB)
```

Figure 9: *As part of ruling out a software/version cause, a routine system package update (apt update && apt upgrade) was run and completed successfully, confirming the fault was configuration-related rather than caused by outdated packages.*

5.2 Evidence

systemd's journal showed the reload consistently failing at the same point, and nginx -t consistently pointed at line 1 of /etc/nginx/nginx.conf — the main configuration file NGINX reads before any site file. Listing /etc/nginx/sites-available and /etc/nginx/sites-enabled showed the group's site file and symlink were present and correctly named, which ruled out the site configuration itself as the cause and pointed at the top-level nginx.conf.

5.3 Investigation

The main configuration file was inspected directly with cat /etc/nginx/nginx.conf. This revealed the file's core structure (events, http blocks, MIME types, logging, gzip and virtual-host include directives) so its content and include paths could be checked against what NGINX expected.

```
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl enable --now nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl status nginx● nginx.service - A high performance web server and a reverse pro
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: 
Active: active (running) since Fri 2026-09-16 18:05:46 +04: 2 days ago
Invocation: 34f0036d92e943cabb5ad83c0d88ae515
Docs: man:nginx(8)
Main PID: 230615 (nginx)
Tasks: 13 (limit: 18537)
Memory: 10M (peak: 28.6M)
CPU: 250ms
CGroup: /system.slice/nginx.service
└─230615 *nginx: master process /usr/sbin/nginx -g daemon on; m
└─230617 *nginx: worker process
└─230618 *nginx: worker process
└─230619 *nginx: worker process
└─230620 *nginx: worker process
└─230621 *nginx: worker process
└─230622 *nginx: worker process
└─230623 *nginx: worker process
└─230624 *nginx: worker process
└─230625 *nginx: worker process
└─230626 *nginx: worker process
└─230627 *nginx: worker process
└─230628 *nginx: worker process

Sep 20 12:27:58 parrot systemd[1]: nginx.service: Control process exited, co
Sep 20 12:27:58 parrot systemd[1]: Reload failed for nginx.service - A high
Sep 20 12:29:15 parrot systemd[1]: Reloading nginx.service - A high performa
Sep 20 12:29:15 parrot nginx[434756]: 2026/09/20 12:29:15 (emerg) 434756#434
Sep 20 12:29:15 parrot systemd[1]: nginx.service: Control process exited, co
Sep 20 12:29:15 parrot systemd[1]: Reload failed for nginx.service - A high
Sep 20 12:31:59 parrot systemd[1]: Reloading nginx.service - A high performa
Sep 20 12:31:59 parrot nginx[435259]: 2026/09/20 12:31:59 (emerg) 435259#435
Sep 20 12:31:59 parrot systemd[1]: nginx.service: Control process exited, co
Sep 20 12:31:59 parrot systemd[1]: Reload failed for nginx.service - A high
... skipping ...
● nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: 
Active: active (running) since Fri 2026-09-16 18:05:46 +04: 2 days ago
Invocation: 34f0036d92e943cabb5ad83c0d88ae515
Docs: man:nginx(8)
Main PID: 230615 (nginx)
Tasks: 13 (limit: 18537)
Memory: 10M (peak: 28.6M)
CPU: 250ms
CGroup: /system.slice/nginx.service
└─230615 *nginx: master process /usr/sbin/nginx -g daemon on; m
└─230617 *nginx: worker process
└─230618 *nginx: worker process
└─230619 *nginx: worker process
└─230620 *nginx: worker process
└─230621 *nginx: worker process
└─230622 *nginx: worker process
└─230623 *nginx: worker process
└─230624 *nginx: worker process
└─230625 *nginx: worker process
└─230626 *nginx: worker process
└─230627 *nginx: worker process
└─230628 *nginx: worker process
```

Figure 10: *NGINX re-enabled and status checked again after the package update; reload failures persisting, confirming the fault was still present in the main configuration file.*

```

Sep 20 12:27:58 parrot systemd[1]: nginx.service: Control process exited, code=exited, status=1/0
Sep 20 12:27:58 parrot systemd[1]: Reload failed for nginx.service - A high performance reverse proxy and web server.
Sep 20 12:29:15 parrot systemd[1]: Reloading nginx.service - A high performance reverse proxy and web server.
Sep 20 12:29:15 parrot systemd[1]: nginx.service: Control process exited, code=exited, status=1/0
Sep 20 12:29:15 parrot systemd[1]: Reload failed for nginx.service - A high performance reverse proxy and web server.
Sep 20 12:31:59 parrot systemd[1]: Reloading nginx.service - A high performance reverse proxy and web server.
Sep 20 12:31:59 parrot nginx(435259): 2025/09/20 12:31:59 (emerg) 435259#435259: *~*~*
Sep 20 12:31:59 parrot systemd[1]: nginx.service: Control process exited, code=exited, status=1/0
Sep 20 12:31:59 parrot systemd[1]: Reload failed for nginx.service - A high performance reverse proxy and web server.

leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ nginx -v
bash: nginx: command not found
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo nginx -t
2025/09/21 10:59:50 (emerg) 548480#548480: getpwnam("webinfra") failed (2: No such file or directory) in /etc/nginx/nginx.conf:1
nginx: configuration file /etc/nginx/nginx.conf test failed
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ cat /etc/nginx/nginx.conf
user webinfra;
worker_processes auto;
worker_cpu_affinity auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 768;
    # multi_accept on;
}

http {
    ##
    # Basic Settings
    ##

    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    # types_hash_bucket_size 64;
    # types_hash_max_size 2048;

    # server_names_hash_bucket_size 64;
    # server_name_in_redirect off;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    ##
    # SSL Settings
    ##

    ssl_protocols TLSv1.2 TLSv1.3; # Dropping SSLv3 (POODLE), TLS 1.0, 1.1
    ssl_prefer_server_ciphers off; # Don't force server cipher order.

    ##
    # Logging Settings
    ##

    access_log /var/log/nginx/access.log;

    ##
    # Gzip Settings
    ##

    gzip on;

    # gzip_vary on;
    # gzip_proxied any;
    # gzip_comp_level 6;
    # gzip_buffers 16 8k;
    # gzip_http_version 1.1;
    # gzip_types text/plain text/css application/json application/javascript text/xml application/xml+rss text/javascript;

    ##
    # Virtual Host Configs
    ##

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}

#mail {
    # See sample authentication script at:
    # http://wiki.nginx.org/imapAuthenticateWithApachePhpScript
    #
    # auth_http localhost/auth.php;
    # pop3_capabilities "TOP" "USER";
    # imap_capabilities "IMAP4rev1" "UIDPLUS";
}

```

Figure 11: Reload-failure log entries alongside the start of `/etc/nginx/nginx.conf`, inspected line by line to check the events and http block structure and include paths.

```

http {
    ##
    # Basic Settings
    ##

    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    server_tokens off; # Recommended practice is to turn this off

    # server_names_hash_bucket_size 64;
    # server_name_in_redirect off;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    ##
    # SSL Settings
    ##

    ssl_protocols TLSv1.2 TLSv1.3; # Dropping SSLv3 (POODLE), TLS 1.0, 1.1
    ssl_prefer_server_ciphers off; # Don't force server cipher order.

    ##
    # Logging Settings
    ##

    access_log /var/log/nginx/access.log;

    ##
    # Gzip Settings
    ##

    gzip on;

    # gzip_vary on;
    # gzip_proxied any;
    # gzip_comp_level 6;
    # gzip_buffers 16 8k;
    # gzip_http_version 1.1;
    # gzip_types text/plain text/css application/json application/javascript text/xml application/xml+rss text/javascript;

    ##
    # Virtual Host Configs
    ##

    include /etc/nginx/conf.d/*.conf;
    include /etc/nginx/sites-enabled/*;
}

#mail {
    # See sample authentication script at:
    # http://wiki.nginx.org/imapAuthenticateWithApachePhpScript
    #
    # auth_http localhost/auth.php;
    # pop3_capabilities "TOP" "USER";
    # imap_capabilities "IMAP4rev1" "UIDPLUS";
}

```

Figure 12: Remainder of `/etc/nginx/nginx.conf` reviewed, including SSL, logging, gzip and the “include `/etc/nginx/sites-enabled/*;`” directive that pulls in the group's site configuration.

5.4 Root Cause

The main `/etc/nginx/nginx.conf` file referenced by the site configuration had an invalid or missing entry point, so NGINX could not parse its configuration tree at all before it ever reached the group's site file.

This is a common infrastructure fault: a single broken top-level include breaks every site behind it, even when each individual site file is syntactically correct.

The main configuration file was corrected with nano, restoring the correct top-level structure and the include `/etc/nginx/sites-enabled/*.conf`; directive. NGINX was validated with `nginx -t`, which returned “syntax is ok” and “test is successful”, then reloaded and re-enabled.

Figure 13: */etc/nginx/nginx.conf corrected in nano; NGINX re-enabled, nginx -t now reports the configuration syntax is ok and the test is successful, and systemctl reload nginx / systemctl status nginx confirm the service is active (running) again.*

A 2048-bit RSA self-signed certificate valid for 365 days was generated with OpenSSL, covering localhost and 127.0.0.1 as subject alternative names. The private key file was restricted to chmod 600 so only root can read it, while the certificate itself is safe to distribute. NGINX was then configured with two server blocks: one listening on port 80 that issues a 301 redirect to HTTPS, and one listening on port 443 that terminates TLS (restricted to TLSv1.2/TLSv1.3, with a hardened cipher list) before proxying to backend_pool exactly as in Part B.

```
server { listen 80; return 301 https://$host$request_uri; }
server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/ssl/nginx-selfsigned.crt;
```

```
ssl_certificate_key /etc/nginx/ssl/nginx-selfsigned.key;
ssl_protocols TLSv1.2 TLSv1.3;
```

```
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo systemctl start backend.service
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ for i in $(seq 1 10); do curl -s http://127.0.0.1; done
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo cp /etc/nginx/sites-available/group07.conf NGINX/nginx_site_configuration.txt
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo cp /etc/nginx/sites-available/group07.conf NGINX/nginx_site_configuration.txt
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t > Evidence/Load_Balancing/nginx_config_test.txt &&
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo mkdir -p /etc/nginx/ssl
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
    -keyout /etc/nginx/ssl/nginx-selfsigned.key \
    -out /etc/nginx/ssl/nginx-selfsigned.crt \
    -subj "/C=US/ST=PortLouis/L=PortLouis/O=WebInfrastructure/OU=Group_07/CN=localhost" \
    -addext "subjaltname=DNS:localhost,IP:127.0.0.1"
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/nginx/ssl/
total 16
-rw-r--r-- 1 root root 1424 Sep 21 11:11 nginx-selfsigned.crt
-rw-r--r-- 1 root root 1708 Sep 21 11:11 nginx-selfsigned.key
-rw-r--r-- 1 root root 1521 Sep 20 12:26 weblnfr.crt
-rw-r--r-- 1 root root 1704 Sep 20 12:26 weblnfr.key
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/ssl/weblnfr.crt
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo rm -f /etc/nginx/ssl/weblnfr.key
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/nginx/ssl/
total 8
-rw-r--r-- 1 root root 1424 Sep 21 11:11 nginx-selfsigned.crt
-rw-r--r-- 1 root root 1708 Sep 21 11:11 nginx-selfsigned.key
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo chmod 600 /etc/nginx/ssl/nginx-selfsigned.key
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ ls -l /etc/nginx/ssl/
total 8
-rw-r--r-- 1 root root 1424 Sep 21 11:11 nginx-selfsigned.crt
-rw-r--r-- 1 root root 1708 Sep 21 11:11 nginx-selfsigned.key
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo tee /etc/nginx/sites-available/group07.conf > /dev/null << EOF
upstream backend_pool {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}

server {
    listen 80;
    server_name localhost;

    return 301 https://$host$request_uri;
}
```

Figure 14: Self-signed certificate and private key generated with OpenSSL (RSA 2048, 365-day validity); key permissions restricted with chmod 600; HTTPS server block written listing the certificate/key paths, TLS protocol and cipher restrictions, and the HTTP→HTTPS redirect.

```
server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate /etc/nginx/ssl/nginx-selfsigned.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx-selfsigned.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://backend_pool;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo systemctl reload nginx
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset:
   Active: active (running) since Fri 2026-09-18 18:05:46 +04; 2 days ago
     Invocation: 14f0036d92e943cbb4ad33c0d89ae516
       Docs: man:nginx(8)
     Process: 551498 ExecReload=/usr/sbin/nginx -g daemon on; master process
    Main PID: 230615 (nginx)
       Tasks: 13 (limit: 18537)
      Memory: 10.6M (peak: 28.6M)
         CPU: 459µs
       CGroup: /system.slice/nginx.service
               └─230615 nginx: master process /usr/sbin/nginx -g daemon on; m
                   └─551497 nginx: worker process
                   └─551498 nginx: worker process
                   └─551499 nginx: worker process
                   └─551500 nginx: worker process
                   └─551501 nginx: worker process
                   └─551502 nginx: worker process
                   └─551503 nginx: worker process
                   └─551504 nginx: worker process
                   └─551505 nginx: worker process
                   └─551506 nginx: worker process
                   └─551507 nginx: worker process
                   └─551508 nginx: worker process

Sep 20 12:31:59 parrot systemd[1]: Reloading nginx.service - A high perform
Sep 20 12:31:59 parrot nginx[435259]: 2026/09/20 12:31:59 (emerg) 435259#435
Sep 20 12:31:59 parrot systemd[1]: nginx.service: Control process exited, co
Sep 20 12:31:59 parrot systemd[1]: Reload failed for nginx.service - A high
Sep 21 11:04:48 parrot systemd[1]: Reloading nginx.service - A high perform
Sep 21 11:04:48 parrot nginx[550901]: 2026/09/21 11:04:48 (notice) 550901#55
Sep 21 11:04:48 parrot systemd[1]: Reloaded nginx.service - A high performan
Sep 21 11:14:30 parrot systemd[1]: Reloading nginx.service - A high perform
Sep 21 11:14:30 parrot nginx[551495]: 2026/09/21 11:14:30 (notice) 551495#55
Sep 21 11:14:30 parrot systemd[1]: Reloaded nginx.service - A high performan
leap_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -I http://127.0.0.1
HTTP/1.1 301 Moved Permanently
```

Figure 15: nginx -t confirms the HTTPS configuration is syntactically valid; NGINX reloaded and confirmed active (running) with the new HTTPS-enabled configuration.

With HTTPS live, `curl -i http://127.0.0.1` returned HTTP/1.1 301 Moved Permanently with a Location: `https://127.0.0.1/` header, confirming the redirect. Repeating the load-balancing test over HTTPS (`curl -sk https://127.0.0.1`, looped ten times) again alternated between Server 1 and Server 2, showing load balancing is preserved under TLS. The certificate was inspected both from disk and as served live by NGINX using `openssl x509` and `openssl s_client`, confirming the subject, issuer, validity window and fingerprint match what was generated. Only the public .crt file was copied into the submission folder; the private key was explicitly excluded.

```
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -i http://127.0.0.1
HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Mon, 21 Sep 2026 07:15:11 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive
Location: https://127.0.0.1/

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<h2><center>nginx</center>
</body>
</html>
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -i http://127.0.0.1 > Evidence/HTTPS/http_redirect_headers.txt
% Total % Received % Xferd Average Speed Time Time Time Current
           %    0 257.2k    0
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -sk http://127.0.0.1
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -sk http://127.0.0.1 > Evidence/HTTPS/http_redirect_final_body.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -sk https://127.0.0.1
Web Infrastructure - Group_07 - Server 1
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ curl -sk https://127.0.0.1
Web Infrastructure - Group_07 - Server 2
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ for i in {1..10}; do curl -sk https://127.0.0.1; done
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ for i in {1..10}; do curl -sk https://127.0.0.1; done > Evidence/HTTPS/https_load_balancing.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ openssl x509 -in /etc/nginx/ssl/nginx-selfsigned.crt -noout -subject -issuer -dates -fingerprint
subject=C=MM, ST=PortLouis, L=PortLouis, O=WebInfrastructure, OU=Group_07, CN=localhost
notBefore=Sep 21 07:11:26 2026 GMT
notAfter=Sep 21 07:11:26 2027 GMT
SHA1 Fingerprint=59:58:C6:70:05:CD:15:EF:49:2A:DC:1C:05:02:F8:89:8A:58:6C:98
X509v3 Subject Alternative Name:
DNS:localhost, IP Address:127.0.0.1
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ openssl s_client -connect 127.0.0.1:443 -servername localhost </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates -fingerprint
subject=C=MM, ST=PortLouis, L=PortLouis, O=WebInfrastructure, OU=Group_07, CN=localhost
notBefore=Sep 21 07:11:26 2026 GMT
notAfter=Sep 21 07:11:26 2027 GMT
SHA1 Fingerprint=59:58:C6:70:05:CD:15:EF:49:2A:DC:1C:05:02:F8:89:8A:58:6C:98
X509v3 Subject Alternative Name:
DNS:localhost, IP Address:127.0.0.1
leo_01@parrot:~/Group_07_SecureWebInfrastructure$ sudo cp /etc/nginx/ssl/nginx-selfsigned.crt TLS/public_certificate_if_used.crt
```

Figure 16: HTTP request to port 80 returns a 301 redirect to HTTPS; repeated HTTPS requests alternate between Server 1 and Server 2; certificate subject/issuer/validity/fingerprint inspected from the certificate file and from the live NGINX TLS connection via `openssl s_client`; public certificate copied to the TLS evidence folder (private key excluded).

7. Part D — Firewall Configuration and Verification

UFW was configured following the principle of least exposure. SSH (22/tcp) was allowed first — before enabling the firewall — to avoid losing remote access; HTTP (80/tcp) and HTTPS (443/tcp) were allowed as the only public web entry points; and the backend ports 3001/tcp and 3002/tcp were explicitly denied, with default policies set to deny all other incoming traffic and allow outgoing traffic.

```
sudo ufw allow 22/tcp comment 'SSH administration'
sudo ufw allow 80/tcp comment 'HTTP redirect to HTTPS'
sudo ufw allow 443/tcp comment 'HTTPS web service'
sudo ufw deny 3001/tcp comment 'Backend 1 internal only'
sudo ufw deny 3002/tcp comment 'Backend 2 internal only'
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw enable
```

```

leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo cp /etc/nginx/sites-enabled/ert TLS/public_certificate_if_used.crt
sudo chown $USER:$USER TLS/public_certificate_if_used.crt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ echo "The TLS private key remains on the Linux server and is not included in this submission"
INCLUDE_PRIVATE_KEY.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw status verbose > Evidence/Firewall/ufw_before.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw status verbose
Status: inactive
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw allow 22/tcp comment 'SSH administration'
Rules updated
Rules updated (v6)
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw allow 80/tcp comment 'HTTP redirect to HTTPS'
Rules updated
Rules updated (v6)
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw allow 443/tcp comment 'HTTPS web service'
Rules updated
Rules updated (v6)
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw deny 3001/tcp comment 'Backend 1 internal only'
sudo ufw deny 3002/tcp comment 'Backend 2 internal only'
Rules updated (v6)
Rules updated
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw default deny incoming
sudo ufw default allow outgoing
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw enable
Firewall is active and enabled on system startup
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
Anywhere on 10
22/tcp ALLOW IN Anywhere # SSH administration
80/tcp ALLOW IN Anywhere # HTTP redirect to HTTPS
443/tcp ALLOW IN Anywhere # HTTPS web service
3001/tcp DENY IN Anywhere # Backend 1 internal only
3002/tcp DENY IN Anywhere # Backend 2 internal only
Anywhere (v6) on 10
22/tcp (v6) ALLOW IN Anywhere (v6) # SSH administration
80/tcp (v6) ALLOW IN Anywhere (v6) # HTTP redirect to HTTPS
443/tcp (v6) ALLOW IN Anywhere (v6) # HTTPS web service
3001/tcp (v6) DENY IN Anywhere (v6) # Backend 1 internal only
3002/tcp (v6) DENY IN Anywhere (v6) # Backend 2 internal only

leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw status verbose > Evidence/Firewall/ufw_after.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw status verbose > Evidence/Firewall/ufw_rules.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ufw status verbose > Evidence/Firewall/ufw_rules.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ hostname -i
192.168.0.233
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ IP=$(hostname -i | awk '{print $1}')

```

Figure 17: Final UFW rule set after enabling the firewall: 22, 80 and 443 allowed from anywhere (IPv4 and IPv6), 3001 and 3002 explicitly denied, and default policy set to deny incoming / allow outgoing.

Testing from the machine's real (non-loopback) IP address confirmed the policy works as intended: HTTP requests returned the expected 301 redirect, HTTPS requests were served successfully, but direct requests to the backend ports failed with “Connection refused”. A final `ss -tlnp` check confirmed the backends still bind only to 127.0.0.1, showing the two layers of protection — loopback binding and firewall rules — are both in effect (defense in depth).

```

leo_01@parrot:~/Group_07_SecureWebInfrastructures$ IP=$(hostname -i | awk '{print $1}')
echo $IP
192.168.0.233
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -I http://$IP
HTTP/1.1 301 Moved Permanently
Server: nginx
Date: Mon, 21 Sep 2026 07:27:00 GMT
Content-Type: text/html
Content-Length: 102
Connection: keep-alive
Location: https://192.168.0.233/

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<h2>nginx</h2>
</body>
</html>
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -k https://$IP
Web Infrastructure - Group 07 - Server 1
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -I http://$IP > Evidence/Firewall/http_allowed.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -I https://$IP > Evidence/Firewall/https_allowed.txt
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -s http://127.0.0.1:3001
curl -s http://127.0.0.1:3002
Web Infrastructure - Group 07 - Server 1
Web Infrastructure - Group 07 - Server 2
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -v http://$IP:3001
* Trying 192.168.0.233 port 3001...
* connect to 192.168.0.233 port 3001 from 192.168.0.233 port 39012 failed: Connection refused
* Failed to connect to 192.168.0.233:3001 after 0 ms: Could not connect to server
* closing connection #0
curl: (7) Failed to connect to 192.168.0.233:3001 after 0 ms: Could not connect to server
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -v http://$IP:3002
* Trying 192.168.0.233:3002...
* connect to 192.168.0.233 port 3002 from 192.168.0.233 port 47914 failed: Connection refused
* Failed to connect to 192.168.0.233:3002 after 0 ms: Could not connect to server
* closing connection #0
curl: (7) Failed to connect to 192.168.0.233:3002 after 0 ms: Could not connect to server
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -v http://$IP:3001 >> Evidence/Firewall/backend3001_blocked.txt || true
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ curl -v http://$IP:3002 >> Evidence/Firewall/backend3002_blocked.txt || true
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo ss -tlnp | grep -E ':3001|:3002' > Evidence/Firewall/backend_listening_loopback.txt
LISTEN 0 5 127.0.0.1:3001 0.0.0.0:* users:(('python3',pid=420295,fd=33))
LISTEN 0 5 127.0.0.1:3002 0.0.0.0:* users:(('python3',pid=535444,fd=33))

leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nano /etc/nginx/sites-available/group07.conf
leo_01@parrot:~/Group_07_SecureWebInfrastructures$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

Figure 18: *External IP discovered; HTTP (301 redirect) and HTTPS requests to that IP succeed, while direct requests to backend ports 3001 and 3002 on the same IP are refused. ss -tlnp confirms the backends remain bound to 127.0.0.1 only, proving the backend ports are not public entry points.*

8. Troubleshooting Case 2 — Backend 2 Excluded From Load Balancing (Deliberate Fault)

8.1 Symptom

A deliberate fault was introduced by adding the backup flag to Backend 2's line in the upstream block: `server 127.0.0.1:3002 backup;`. After reloading NGINX, ten repeated requests over HTTPS returned only “Server 1” — Backend 2 appeared to have stopped participating in load balancing entirely.

8.2 Evidence

Both `backend1.service` and `backend2.service` were confirmed active, and both ports were confirmed listening. A direct curl to Backend 2 on `127.0.0.1:3002` succeeded, proving the service itself was healthy. Grepping the live NGINX configuration for the upstream `backend_pool` block showed the `backup` keyword present on the Backend 2 line — the actual, resolved configuration NGINX was running with, not just the file on disk.

8.3 Investigation

To confirm the hypothesis, `backend1.service` was stopped and the request loop repeated. Backend 2 immediately began responding, which confirms it was healthy and reachable but was only being used as a standby — consistent with NGINX's documented behaviour that a server marked `backup` only receives traffic once all non-backup servers in the pool are unavailable. Backend 1 was then restarted.

8.4 Root Cause

The `backup` directive on the Backend 2 upstream line removed it from normal round-robin rotation, so under regular operation NGINX only ever selected Backend 1. This is a configuration-level fault, not a backend or network failure — both services were fully functional throughout.

8.5 Fix and Verification

The `backup` keyword was removed from the upstream block, restoring both servers to equal standing. The configuration was validated with `nginx -t` and reloaded. Ten repeated HTTPS requests afterward again alternated between Server 1 and Server 2, confirming load balancing was fully restored. Final service checks showed `nginx.service` active (running); `backend2.service` active (running); `backend1.service` had briefly failed and auto-restarted under its `Restart=on-failure` policy during testing, then returned to a healthy running state, consistent with the resiliency built into the `systemd` unit files in Part A.

```

nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl reload nginx
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo ss -tlnp | grep -E ':(3001|3002)' > Evidence/Troubleshooting/backend_listening_loopback.txt
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ for i in {1..10}; do curl -sk https://127.0.0.1; done
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ for i in {1..10}; do curl -sk https://127.0.0.1; done > Evidence/Troubleshooting/1_symptom_o
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ systemctl is-active backend1.service backend2.service > Evidence/Troubleshooting/2_backend_s
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo ss -tlnp | grep -E ':(3001|3002)' > Evidence/Troubleshooting/2_backend_listeners.txt
curl -s https://127.0.0.1:3002 > Evidence/Troubleshooting/2_backend2_direct.txt
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo nginx -t | grep -A3 'upstream backend_pool' > Evidence/Troubleshooting/2_nginx_upstream
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl stop backend1.service
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ for i in {1..5}; do curl -sk https://127.0.0.1; done > Evidence/Troubleshooting/3_backend1_s
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ for i in {1..5}; do curl -sk https://127.0.0.1; done
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl start backend1.service
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo nginx -t | grep -A3 'upstream backend_pool'
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
server 127.0.0.1:3001:
server 127.0.0.1:3002 backup;
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo nano /etc/nginx/sites-available/groupxx.conf
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo nano /etc/nginx/sites-available/group07.conf
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl reload nginx
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ for i in {1..10}; do curl -sk https://127.0.0.1; done > Evidence/Troubleshooting/5_fix_verif
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ for i in {1..10}; do curl -sk https://127.0.0.1; done
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 2
Web Infrastructure - Group_07 - Server 1
leo_01@parrot: ~/Group_07_SecureWebInfrastructures$ sudo systemctl status backend1.service
sudo systemctl status backend2.service
sudo systemctl status nginx

```

Figure 19: Fault introduced (server ... backup;), reloaded, and reproduced: repeated requests return only Server 1. Evidence gathered — both backend services active, both ports listening, Backend 2 reachable directly, and the live upstream block confirmed to contain the backup flag. Backend 1 stopped to test the hypothesis; requests then begin returning Server 2, confirming it as a standby-only member. backup removed and configuration re-validated.

```

* backend1.service - Web Infrastructure Backend 1
Loaded: loaded (/etc/systemd/system/backend1.service; enabled; preset:
Active: active (running) since Mon 2026-09-21 10:37:56 +04; 58min ago
Invocation: cce389645965472db7e0d9111e549592
Main PID: 535444 (python3)
Tasks: 1 (limit: 16537)
Memory: 8.6M (peak: 9.4M)
CPU: 605ms
CGroup: /system.slice/backend1.service
└─535444 /usr/bin/python3 /opt/backend1/server.py

Sep 21 11:17:15 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:17:15)
Sep 21 11:17:15 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:17:15)
Sep 21 11:17:31 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:17:31)
Sep 21 11:17:35 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:17:35)
Sep 21 11:27:00 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:27:00)
Sep 21 11:32:41 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:32:41)
Sep 21 11:35:51 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:35:51)
Sep 21 11:35:51 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:35:51)
Sep 21 11:35:51 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:35:51)
Sep 21 11:35:51 parrot python3(535444): 127.0.0.1 - - (21/Sep/2026 11:35:51)
* nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset:
Active: active (running) since Fri 2026-09-18 18:05:46 +04; 2 days ago
Invocation: 34f0b3d092e943cab8ad33c0d88ae515
Docs: man:nginx(8)
Process: 555578 ExecReload=/usr/sbin/nginx -g daemon on; master process
Main PID: 230615 (nginx)
Tasks: 13 (limit: 16537)
Memory: 14M (peak: 20M)
CPU: 896ms
CGroup: /system.slice/nginx.service
└─230615 'nginx: master process /usr/sbin/nginx -g daemon on; m
└─555579 'nginx: worker process'
└─555580 'nginx: worker process'
└─555581 'nginx: worker process'
└─555582 'nginx: worker process'
└─555583 'nginx: worker process'
└─555584 'nginx: worker process'
└─555585 'nginx: worker process'
└─555586 'nginx: worker process'
└─555587 'nginx: worker process'
└─555588 'nginx: worker process'
└─555589 'nginx: worker process'
└─555590 'nginx: worker process'
└─555591 'nginx: worker process'

Sep 21 11:04:48 parrot systemd[1]: Reloaded nginx.service - A high performan
Sep 21 11:14:30 parrot systemd[1]: Reloaded nginx.service - A high performan
Sep 21 11:14:30 parrot nginx(55169): 2026/09/21 11:14:30 [notice] 55169#55
Sep 21 11:14:30 parrot systemd[1]: Reloaded nginx.service - A high performan
Sep 21 11:31:06 parrot systemd[1]: Reloaded nginx.service - A high performan
Sep 21 11:31:06 parrot nginx(554540): 2026/09/21 11:31:06 [notice] 554540#5
Sep 21 11:31:06 parrot systemd[1]: Reloaded nginx.service - A high performan

```

Figure 20: Fix verified: ten repeated HTTPS requests again alternate between Server 1 and Server 2. backend1.service status shows a transient failure and automatic restart (Restart=on-failure) during testing before settling into active (running); backend2.service and nginx.service both confirmed active (running).

9. Part F — Architecture and Security Reasoning

Why route clients through NGINX instead of the backends directly?

NGINX is the single public entry point, giving one place to terminate TLS, apply the firewall boundary, load-balance, and swap or scale backends without any client-visible change. Direct backend access would bypass every one of those controls.

What does HTTPS add over HTTP?

Confidentiality (encryption), integrity (tamper detection) and server authentication via the certificate. HTTP is clear-text and can be read or altered in transit.

Why keep a firewall if the backends already bind to loopback only?

Defense in depth: loopback binding and firewall rules are independent controls. If one is ever misconfigured — a backend accidentally bound to 0.0.0.0, for example — the other still blocks external access, as demonstrated in Section 7.

How do two backends improve availability?

Both Section 4 and Section 8 demonstrated this directly: stopping one backend did not take the service down, because NGINX continued routing to the remaining healthy backend.

10. Conclusion

Across this exercise, TLS secured client traffic, the UFW firewall enforced least-exposure network access, NGINX supplied a single hardened entry point with load balancing across two independently managed backends, and two real troubleshooting incidents — a broken top-level NGINX configuration and a misapplied backup directive — were diagnosed using the same evidence-led method: reproduce the symptom, gather evidence, investigate hypotheses, isolate the root cause, apply a minimal fix, and verify recovery. Together these mechanisms and the two case studies demonstrate a more secure, available and maintainable production-style web environment than a single exposed backend server would provide.

11. Sources and AI Transparency

AI assistance was used to help plan the assignment structure, interpret NGINX/UFW/OpenSSL output, and compile this report from the group's own command output and screenshots captured on the group's Parrot OS environment. All configuration, execution, testing and troubleshooting shown in the figures above was carried out by the group on its own system.